Beginner Guide to SRE Error Budgets and Reliability

Error budgets give software teams a clear, data-driven rule to balance building new features against maintaining uptime. Mastering error budgets helps junior engineers deploy code safely without risking major customer-facing outages.

What you'll learn:

1. Measuring Live Health with SLIs
2. Calculating Monthly Error Budgets
3. Tracking Success Rates in Prometheus
4. Automating Feature Freezes in CI/CD

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Measuring Live Health with SLIs

A Service Level Indicator (SLI) is a direct measurement of your application's behavior over time. You calculate it by dividing successful requests by the total number of requests. Running this curl command lets you inspect HTTP status codes and response latency directly from your terminal.

```
$curl -w "Status: %{http_code} | Time: %{time_total}s\n" -o /dev/null -s https://httpbin.org/status/2
```

SRE Error Budgets: How to Balance Features vs Reliability

Monday, October 26, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Calculating Monthly Error Budgets

An Error Budget is simply 100% minus your Service Level Objective (SLO). If your target is 99.9% availability, your error budget is 0.1% allowed downtime per month. Use this quick Python command to calculate exact allowed downtime minutes for any SLO.

```
$python3 -c "slo=99.9; mins=30*24*60; downtime=(100-slo)/100*mins; print(f'Allowed Downtime: {downtim
```

SRE Error Budgets: How to Balance Features vs Reliability

Monday, October 26, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Tracking Success Rates in Prometheus

Prometheus uses PromQL to aggregate metric data from your microservices in real time. By calculating the rate of 2xx success responses against total requests, you get an exact SLI percentage. This query helps you plot live app stability on Grafana dashboards.

```
$sum(rate(http_requests_total{status=~"2.."}[5m])) / sum(rate(http_requests_total[5m])) * 100
```

SRE Error Budgets: How to Balance Features vs Reliability

Monday, October 26, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Automating Feature Freezes in CI/CD

When your error budget drops to zero, your CI/CD pipeline should block feature releases until system stability is restored. This bash snippet checks remaining budget percentage and halts deployment scripts automatically if the budget is exhausted.

```
$BUDGET=$(curl -s http://monitoring.local/api/v1/budget); if [ "$BUDGET" -le 0 ]; then echo "Budget s
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Start with easy reliability targets (like 99%) before aiming for 99.9%.
+ Treat your error budget as a budget to spend on innovation, not failure.
+ Set alerts when 50% and 80% of your monthly budget is spent.
+ Enforce automatic feature freezes when the error budget hits zero.
+ Review budget burn rates during weekly team operational syncs.

Watch Out For

! Setting a 100% uptime goal, which makes shipping new code nearly impossible.
! Ignoring burn rate alerts until the error budget is completely gone.
! Using error budgets to blame developers instead of fixing system flaws.
! Measuring SLIs from internal networks instead of actual user endpoints.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh